HDOC - Day 4
(C Programming)

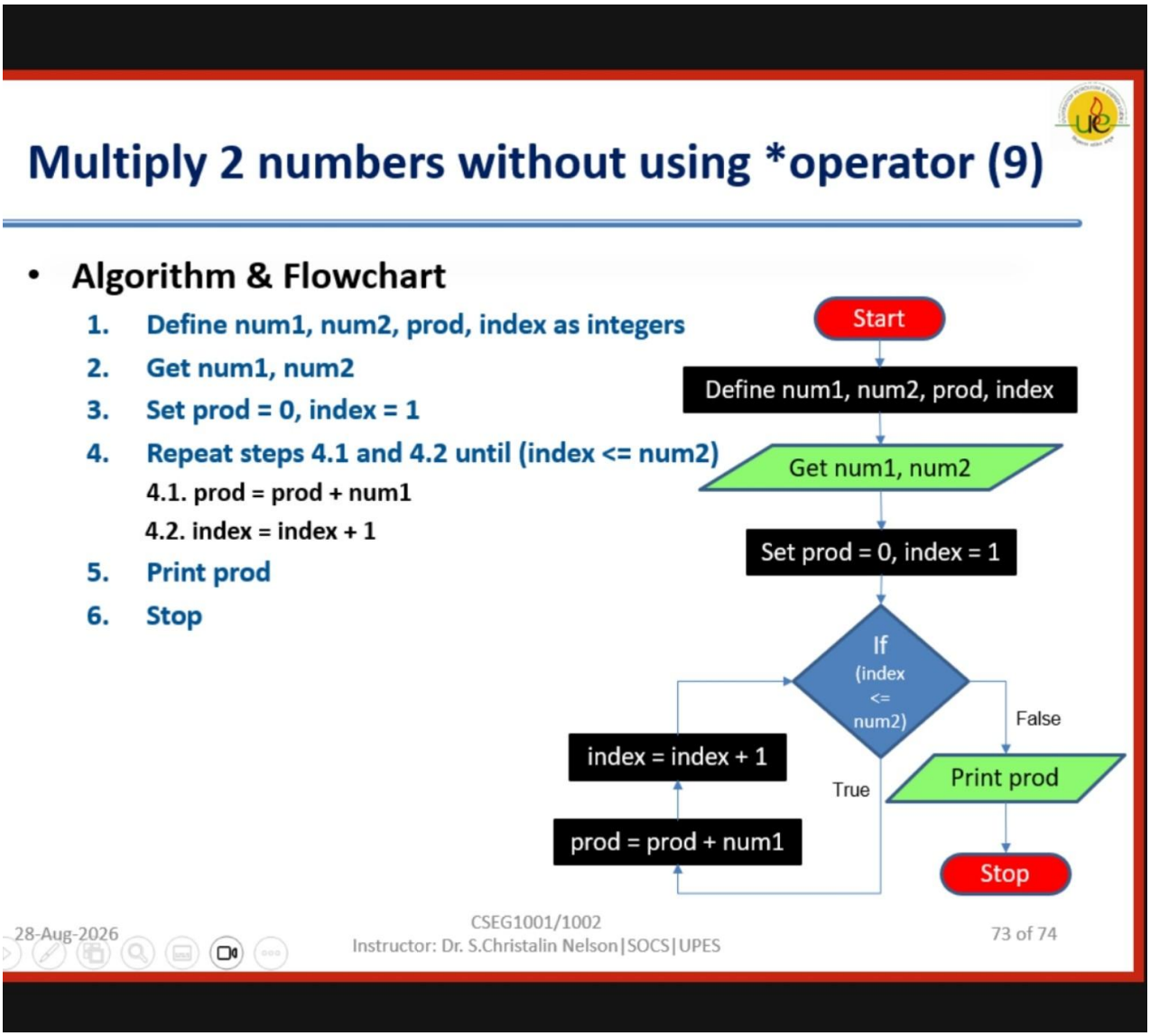
Multiplication of Two Integers Without Using the * Operator

1. Problem Statement

Find the product of two integers without using the multiplication (*) operator. Use repeated addition, use the minimum possible number of additions, and correctly handle positive, negative and zero inputs.

2. Given Incomplete Algorithm and Flowchart

The following algorithm and flowchart were provided for the activity. They use num2 directly in the loop condition and therefore do not satisfy all the stated requirements.



3. Mistakes in the Given Algorithm

1. **Loop count is fixed to num2:** The condition index <= num2 makes num2 control the number of repeated additions. This does not always give the minimum number of additions. For input 3 and 12, it would repeat 12 times instead of 3.
2. **Negative values are not handled correctly:** If num2 is negative, the loop condition may fail immediately, so the required repeated addition is not performed.
3. **Sign of the product is not handled:** The algorithm does not separately determine whether the final product should be positive or negative.
4. **Zero is not handled explicitly:** A zero input should immediately give product 0 with 0 additions.
5. **Absolute values are not used:** The smaller absolute value must be used as the repetition count so that negative inputs and minimum additions are handled correctly.

4. Corrected Algorithm

1. Start.

2. Input two integers num1 and num2.

3. Set product = 0.

4. If num1 = 0 OR num2 = 0, print product = 0 and additions = 0, then Stop.

5. Set x = |num1| and y = |num2|.

6. If x < y, set count = x and addend = y. Otherwise, set count = y and addend = x.

7. Set i = 0.

8. While i < count: set product = product + addend, then set i = i + 1.

9. If num1 and num2 have different signs, set product = -product.

10. Print product and additions = count.

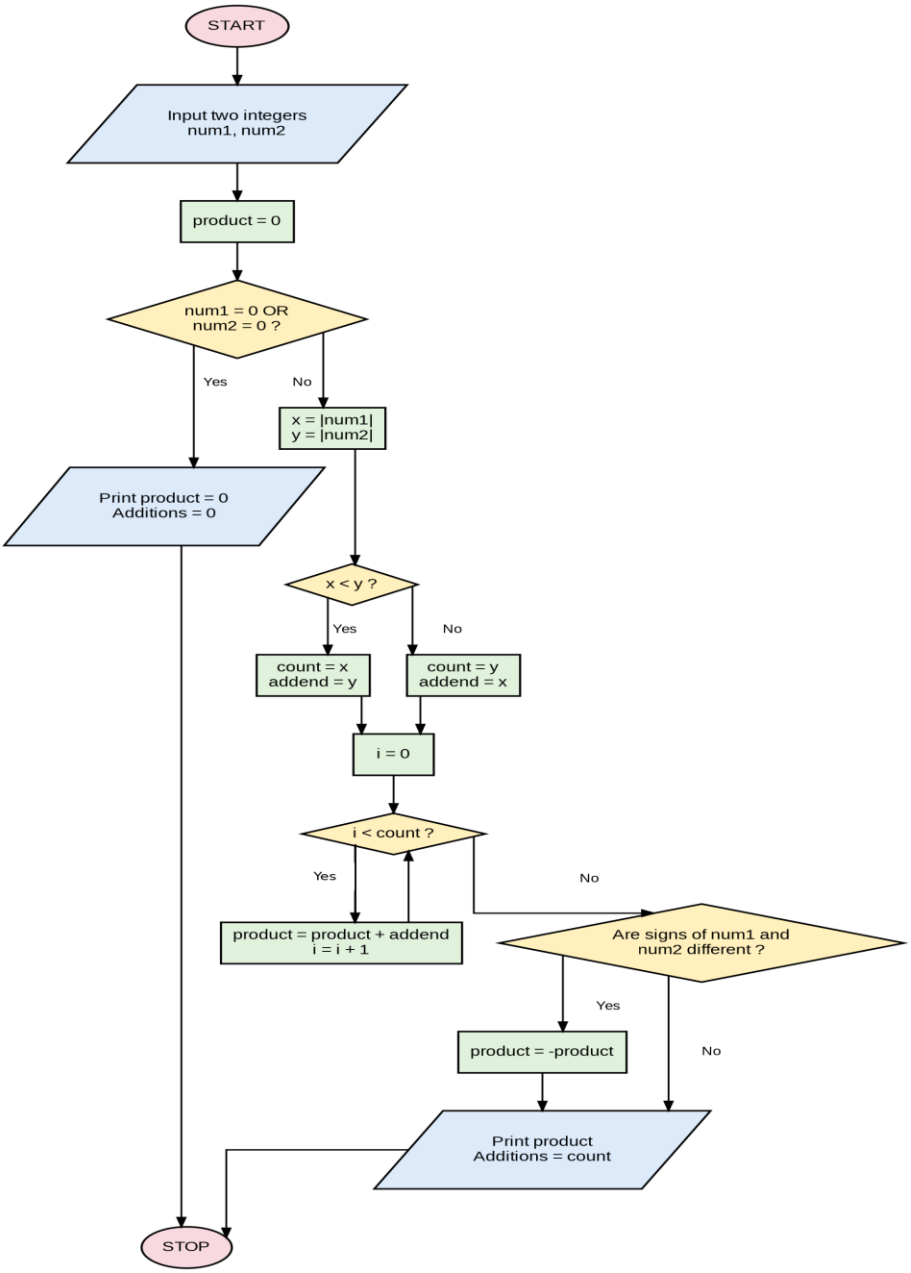
11. Stop.

5. Mistakes in the Given Flowchart

- The decision diamond uses index <= num2, so num2 is always treated as the loop count.
- There is no comparison of |num1| and |num2| to choose the smaller value as the repetition count.
- There is no explicit check for multiplication by zero.
- There is no proper handling of negative operands or the final sign of the product.
- Therefore, the flowchart does not satisfy all 13 supplied test cases or the minimum-additions requirement.

6. Corrected Flowchart

The corrected flowchart uses count as the loop limit, where count is the smaller absolute value. The variable i is the loop counter and addend is the value repeatedly added.



7. Verification Using the Given Test Cases

The corrected algorithm and flowchart are checked by dry-running each input and comparing the obtained product and number of additions with the values given in the assignment.

Test	Input	Expected	Obtained	Additions	Result
1	12, 3	36	36	3	PASS
2	3, 12	36	36	3	PASS
3	-6, 5	-30	-30	5	PASS
4	6, -5	-30	-30	5	PASS
5	-9, -4	36	36	4	PASS
6	-4, -9	36	36	4	PASS
7	0, 8	0	0	0	PASS
8	8, 0	0	0	0	PASS
9	0, -7	0	0	0	PASS
10	-7, 0	0	0	0	PASS
11	0, 0	0	0	0	PASS
12	1, -7	-7	-7	1	PASS
13	-1, -9	9	9	1	PASS

7. Sample Dry Runs

Example 1: Input 12 and 3

- Both numbers are positive.
- Since **12 > 3**, set **count = 3** and **addend = 12**.
- Product changes as:
    - **0 + 12 = 12**
    - **12 + 12 = 24**
    - **24 + 12 = 36**
- The signs are the same, so the product remains **36**.
- **Product = 36; additions = 3.**

Example 2: Input -6 and 5

- Convert **-6** to **6** and record a negative sign.
- Set **count = 5** and **addend = 6**.
- Repeated addition gives:
    - **6 + 6 + 6 + 6 + 6 = 30**
- Since the signs are different, apply the negative sign.
- **Product = -30; additions = 5.**

Example 3: Input -9 and -4

- Convert both numbers to positive: **9** and **4**.
- Set **count = 4** and **addend = 9**.
- Repeated addition gives:
    - **9 + 9 + 9 + 9 = 36**
- Both inputs have the same sign, so the result remains positive.
- **Product = 36; additions = 4.**

Example 4: Input 0 and -7

- The zero condition is true.
- Set **product = 0**.
- No repeated addition is performed.
- **Product = 0; additions = 0.**

8. Conclusion

The corrected algorithm and flowchart multiply two integers without using the * operator. They correctly handle all sign combinations and zero, use the smaller absolute value as the loop count, and satisfy all 13 supplied test cases with the required minimum number of additions.